

软件安全作业一

班级：2022211801 学号：2022211570 姓名：项枫

一、常见的字符串操作错误以及缓解措施

1、无界字符串复制

发生于从源数据复制数据到一个定长的字符数组时。从无界数据源(例如 `stdin`)读入数据，由于事先无法得知用户将会输入多少个字符，因此不可能预先分配一个长度足够的数组。

缓解措施：静态分配一个认为长度远远大于所需的数组，不要从一个无界源复制数据到定长数组，不要使用废弃或过时的函数。

2、空结尾错误

一个字符串正确地以空字符结尾，是指在数组最后一个元素处或在它之前存在一个空终止符。如果一个字符串没有以空字符结尾，程序可能会被欺骗，导致在数组边界之外读取或写入数据。

缓解措施：字符串必须在数组的最后一个元素的地址处或在它之前包含一个空终止字符，才可以安全地作为标准字符串处理函数如 `strcpy()` 函数或 `strlen()` 函数的参数被传递。空终止字符之所以是必要的，是因为前面这些函数以及其它由 C 标准定义的字符串处理函数，都依赖于它的存在来标记字符串的结尾。同样，如果程序对一个字符数组迭代循环的终止条件取决于为字符串分配的内存是否存在一个空终止字符，字符串也必须以空字符结尾。按要求提供空字节结尾的字符串。

3、截断

当目标字符数组的长度不足以容纳一个字符串的内容时，就会发生字符串截断。

缓解措施：截断通常发生于读取用户输入或字符串复制时，通常是程序员试图防止缓冲区溢出的结果。

4、差一错误

与无界字符串复制有相似之处，即都涉及对数组的越界写问题。

二、编写函数

1、函数编写思路 and 流程

(1) 安全 strcpy 函数

参数验证：首先验证输入参数是否为有效指针，确保目标和源指针都不为 NULL。其次，验证目标缓冲区的大小是否有效，不能为 0。

长度检查：获取源字符串的长度，通过 `strlen` 计算源字符串的长度。比较源字符串的长度和目标缓冲区的大小(`s1max`)。如果源字符串的长度大于等于目标缓冲区的大小，说明目标缓冲区无法容纳源字符串及其末尾的 `\0`，需要返回 `ERANGE` 错误并清空目标缓冲区。

字符串复制：如果长度检查通过，则使用 `for` 循环将源字符串逐字符复制到目标缓冲区，并确保目标字符串以 `\0` 结尾。

返回值：如果复制成功，则返回 0；如果有错误发生，如指针无效或缓冲区过小，则返回相应的错误码（如 `EINVAL` 或 `ERANGE`）。

(2) 测试函数

测试目标：测试 `strcpy_s` 函数的正确性和健壮性，确保它能处理不同的输入情况，包括有效和无效的输入。

正常测试用例：输入一个小于目标缓冲区大小的字符串，验证函数能正确复制字符串并返回成功。

错误测试用例：1) 无界的字符串：传递一个无界的源字符串，验证函数能正确检测并返回 `ERANGE` 错误，同时确保目标缓冲区被清空。2) 空结尾的字符串：传递一个空结尾的源字符串，验证函数能正确检测并返回 `ERANGE` 错误，同时确保目标缓冲区被清空。3) 截断的字符串：传递一个超过目标缓冲区大小的源字符串，验证函数能正确检测并返回 `ERANGE` 错误，同时确保目标缓冲区被清空。4) 过长的字符串：传递一个超过目标缓冲区大小的源字符串，验证函数能正确检测并返回 `ERANGE` 错误，同时确保目标缓冲区被清空。5) 空字符串：传递 `NULL` 作为源字符串，验证函数返回 `EINVAL` 错误。

结果验证：每个测试用例后，通过 `strcmp` 检查目标缓冲区内容是否与预期一致，并输出结果（成功或错误码）。

2、函数源代码

（1）安全 `strcpy` 函数

```
// 安全 strcpy 函数定义
errno_t strcpy_s(char *restrict s1, size_t s1max, const char *restrict s2) {
    if (s1 == NULL || s2 == NULL) {
        return EINVAL; // 无效的参数
    }
    if (s1max == 0) {
        return ERANGE; // 缓冲区大小无效
    }

    size_t src_len = strlen(s2);

    if (src_len >= s1max) {
        s1[0] = '\0'; // 清空目标缓冲区
        return ERANGE; // 缓冲区过小
    }

    // 复制字符串并确保以 \0 结尾
    for (size_t i = 0; i <= src_len; ++i) {
        s1[i] = s2[i];
    }

    return 0; // 成功复制
}
```

（2）测试函数

```
// 测试函数
void test_strcpy_s() {

    char dest[10]; // 目标缓冲区
    errno_t result;

    // 成功测试
    const char *source1 = "Hello"; // 合适的长度

    result = strcpy_s(dest, sizeof(dest), source1);
```

```

printf("test1:\n");
printf("source is %s\n", source1);

if (result == 0 && strcmp(dest, source1) == 0) {
    printf("Success!\n");
}
else {
    printf("Error: %d, ", result);

    if (result == 34) printf("ERANGE\n");
    else if (result == 22) printf("EINVAL\n");
}

printf("\n");

//错误测试 - 无界的字符串
const char *source2 = "01234567890123456789bupt"; // 无界的字符串

result = strcpy_s(dest, sizeof(dest), source2);

printf("test2:\n");
printf("source is %s\n", source2);

printf("Error: %d, ", result);

if (result == 34) printf("ERANGE\n");
else if (result == 22) printf("EINVAL\n");

printf("\n");

//错误测试 - 空结尾的字符串
const char *source3 = "0123456789";

result = strcpy_s(dest, sizeof(dest), source3);

printf("test3:\n");
printf("source is %s\n", source3);

printf("Error: %d, ", result);

if (result == 34) printf("ERANGE\n");
else if (result == 22) printf("EINVAL\n");

```

```

printf("\n");

//错误测试 - 意外截断字符串
const char *source4 = "0123456789buptbupt";

result = strcpy_s(dest, sizeof(dest), source4);

printf("test4:\n");
printf("source is %s\n", source4);

printf("Error: %d, ", result);

if (result == 34) printf("ERANGE\n");
else if (result == 22) printf("EINVAL");

printf("\n");

//错误测试 - 过长的字符串
const char *source5 = "buptbuptbuptbuptbupt";

result = strcpy_s(dest, sizeof(dest), source4);

printf("test5:\n");
printf("source is %s\n", source5);

printf("Error: %d, ", result);

if (result == 34) printf("ERANGE\n");
else if (result == 22) printf("EINVAL\n");

printf("\n");

//错误测试 - 源字符串为 NULL
result = strcpy_s(dest, sizeof(dest), NULL);

printf("test6:\n");
printf("source is %s\n", NULL);

printf("Error: %d, ", result);

```

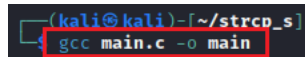
```
if (result == 34) printf("ERANGE\n");
else if (result == 22) printf("EINVAL\n");

}
```

3、运行结果

(1) 编译

```
gcc main.c -o main
```



(2) 运行

```
./main
```

test1: 正常测试用例，输入一个小于目标缓冲区大小的字符串，验证函数能正确复制字符串并返回成功。

test2: 无界的字符串，传递一个无界的源字符串，验证函数能正确检测并返回 ERANGE 错误，同时确保目标缓冲区被清空。

test3: 空结尾的字符串，传递一个空结尾的源字符串，验证函数能正确检测并返回 ERANGE 错误，同时确保目标缓冲区被清空。

test4: 截断的字符串，传递一个超过目标缓冲区大小的源字符串，验证函数能正确检测并返回 ERANGE 错误，同时确保目标缓冲区被清空。

test5: 过长的字符串，传递一个超过目标缓冲区大小的源字符串，验证函数能正确检测并返回 ERANGE 错误，同时确保目标缓冲区被清空。

test6: 空字符串，传递 NULL 作为源字符串，验证函数返回 EINVAL 错误。

```
(kali@kali)-[~/strcp_s]  
$ ./main
```

```
test1:  
source is Hello  
Success!
```

```
test2:  
source is 01234567890123456789bupt  
Error : 34, ERANGE
```

```
test3:  
source is 0123456789  
Error : 34, ERANGE
```

```
test4:  
source is 0123456789buptbupt  
Error : 34, ERANGE
```

```
test5:  
source is buptbuptbuptbuptbupt  
Error : 34, ERANGE
```

```
test6:  
source is (null)  
Error : 22, EINVAL
```